Exercise – 2

Name: Sparsh Karna

Registration Number: 23BDS1172

1. Server

```
import socket



# S-DES tables and permutations (0-indexed, converted from standard 1-indexed)
P10 = [2, 4, 1, 6, 3, 9, 0, 8, 7, 5] # Standard: 3,5,2,7,4,10,1,9,8,6
P8 = [5, 2, 6, 3, 7, 4, 9, 8] # Standard: 6,3,7,4,8,5,10,9
IP = [1, 5, 2, 0, 3, 7, 4, 6] # Standard: 2,6,3,1,4,8,5,7
IP_INV = [3, 0, 2, 4, 6, 1, 7, 5] # Standard: 4,1,3,5,7,2,8,6
EP = [3, 0, 1, 2, 1, 2, 3, 0] # Standard: 4,1,2,3,2,3,4,1
P4 = [1, 3, 2, 0] # Standard: 2,4,3,1

S0 = [
[1, 0, 3, 2],
[3, 2, 1, 0],
[0, 2, 1, 3],
[3, 1, 3, 2]
]

S1 = [
[0, 1, 2, 3],
[2, 0, 1, 3],
[3, 0, 1, 0],
[2, 1, 0, 3]
]



def table(input_bits, key, n):
return [input_bits[key[i]] for i in range(n)]



def leftshift(a, n, x):
for _ in range(x):
a[:] = a[1:] + [a[0]]
```

```
def split(input_bits, n):
mid = n // 2
return input_bits[:mid], input_bits[mid:]


def merge(l, r):
return l + r


def xor_bits(a, b, n):
return [a[i] ^ b[i] for i in range(n)]


def keygen(key):
temp = table(key, P10, 10)
l, r = split(temp, 10)
leftshift(l, 5, 1)
leftshift(r, 5, 1)
temp = merge(l, r)
k1 = table(temp, P8, 8)
leftshift(l, 5, 2)
leftshift(r, 5, 2)
temp = merge(l, r)
k2 = table(temp, P8, 8)
return k1, k2


def sbox(input_bits, s):
r = input_bits[0] * 2 + input_bits[3]
c = input_bits[1] * 2 + input_bits[2]
val = s[r][c]
return [(val >> 1) & 1, val & 1]


def round_func(text, k):
L, R = split(text, 8)
temp = table(R, EP, 8)
temp = xor_bits(temp, k, 8)
l, r = split(temp, 8)
o1 = sbox(l, S0)
o2 = sbox(r, S1)
t = o1 + o2
p4out = table(t, P4, 4)
p4out = xor_bits(p4out, L, 4)
# Return [R, p4out] as per C++ code: text[i] = R[i], text[i+4] = p4out[i]
return R + p4out
```

```
def swap_halves(a):
return a[4:] + a[:4]



def sdes_encrypt(text, key):
k1, k2 = keygen(key)
im = table(text, IP, 8)
im = round_func(im, k1)
im = swap_halves(im)
im = round_func(im, k2)
# Final swap to undo the implicit swap in round_func
im = swap_halves(im)
result = table(im, IP_INV, 8)
return result



def bits_to_string(bits):
return ''.join(map(str, bits))



def string_to_bits(s):
return [int(b) for b in s]



def main():
plaintext = input("Enter 8-bit plaintext (e.g., 10101010): ")
key = input("Enter 10-bit key (e.g., 1010000010): ")
# Validate input
if len(plaintext) != 8 or not all(c in '01' for c in plaintext):
print("Error: Plaintext must be exactly 8 bits (0s and 1s)")
return
if len(key) != 10 or not all(c in '01' for c in key):
print("Error: Key must be exactly 10 bits (0s and 1s)")
return
plaintext_bits = string_to_bits(plaintext)
key_bits = string_to_bits(key)
encrypted = sdes_encrypt(plaintext_bits, key_bits)
encrypted_str = bits_to_string(encrypted)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(('localhost', 5000))
server_socket.listen(1)
print("Server started. Waiting for client...")
conn, addr = server_socket.accept()
print("Client connected.")
conn.sendall((encrypted_str + "\n").encode())
print("Encrypted message sent:", encrypted_str)
conn.close()
server_socket.close()
```

```
if __name__ == "__main__":
main()
```

2. Client

```
import socket



# S-DES tables and permutations (0-indexed, converted from standard 1-indexed)
P10 = [2, 4, 1, 6, 3, 9, 0, 8, 7, 5] # Standard: 3,5,2,7,4,10,1,9,8,6
P8 = [5, 2, 6, 3, 7, 4, 9, 8] # Standard: 6,3,7,4,8,5,10,9
IP = [1, 5, 2, 0, 3, 7, 4, 6] # Standard: 2,6,3,1,4,8,5,7
IP_INV = [3, 0, 2, 4, 6, 1, 7, 5] # Standard: 4,1,3,5,7,2,8,6
EP = [3, 0, 1, 2, 1, 2, 3, 0] # Standard: 4,1,2,3,2,3,4,1
P4 = [1, 3, 2, 0] # Standard: 2,4,3,1

S0 = [
[1, 0, 3, 2],
[3, 2, 1, 0],
[0, 2, 1, 3],
[3, 1, 3, 2]
]

S1 = [
[0, 1, 2, 3],
[2, 0, 1, 3],
[3, 0, 1, 0],
[2, 1, 0, 3]
]



def table(input_bits, key, n):
return [input_bits[key[i]] for i in range(n)]



def leftshift(a, n, x):
for _ in range(x):
a[:] = a[1:] + [a[0]]



def split(input_bits, n):
mid = n // 2
return input_bits[:mid], input_bits[mid:]
```

```
def merge(l, r):
return l + r


def xor_bits(a, b, n):
return [a[i] ^ b[i] for i in range(n)]


def keygen(key):
temp = table(key, P10, 10)
l, r = split(temp, 10)
leftshift(l, 5, 1)
leftshift(r, 5, 1)
temp = merge(l, r)
k1 = table(temp, P8, 8)
leftshift(l, 5, 2)
leftshift(r, 5, 2)
temp = merge(l, r)
k2 = table(temp, P8, 8)
return k1, k2


def sbox(input_bits, s):
r = input_bits[0] * 2 + input_bits[3]
c = input_bits[1] * 2 + input_bits[2]
val = s[r][c]
return [(val >> 1) & 1, val & 1]


def round_func(text, k):
L, R = split(text, 8)
temp = table(R, EP, 8)
temp = xor_bits(temp, k, 8)
l, r = split(temp, 8)
o1 = sbox(l, S0)
o2 = sbox(r, S1)
t = o1 + o2
p4out = table(t, P4, 4)
p4out = xor_bits(p4out, L, 4)
# Return [R, p4out] as per C++ code: text[i] = R[i], text[i+4] = p4out[i]
return R + p4out


def swap_halves(a):
return a[4:] + a[:4]


def sdes_decrypt(text, key):
```

```
k1, k2 = keygen(key)
im = table(text, IP, 8)
im = round_func(im, k2)
im = swap_halves(im)
im = round_func(im, k1)
# Final swap to undo the implicit swap in round_func
im = swap_halves(im)
result = table(im, IP_INV, 8)
return result



def bits_to_string(bits):
return ''.join(map(str, bits))



def string_to_bits(s):
return [int(b) for b in s]



def main():
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 5000))
encrypted = client_socket.recv(1024).decode().strip()
print("Encrypted message received:", encrypted)
key = input("Enter 10-bit key (e.g., 1010000010): ")
# Validate input
if len(key) != 10 or not all(c in '01' for c in key):
print("Error: Key must be exactly 10 bits (0s and 1s)")
client_socket.close()
return
encrypted_bits = string_to_bits(encrypted)
key_bits = string_to_bits(key)
decrypted = sdes_decrypt(encrypted_bits, key_bits)
decrypted_str = bits_to_string(decrypted)
print("Decrypted (original) message:", decrypted_str)
client_socket.close()



if __name__ == "__main__":
main()
```

3. Output

```
❯ python3 server.py
Enter 8-bit plaintext (e.g., 10101010): 10101010
Enter 10-bit key (e.g., 1010000010): 1010101010
Server started. Waiting for client...
Client connected.
Encrypted message sent: 11101010

~/dev/sem6/crypto-lab/lab2/sdes main
❯
```

sdes — sparshkarna@Sparshs-MacBook-Air — ..lab/

```
❯ python3 client.py
Encrypted message received: 11101010
Enter 10-bit key (e.g., 1010000010): 1010101010
Decrypted (original) message: 10101010

~/dev/sem6/crypto-lab/lab2/sdes main
❯
```